SC42101 Networked and Distributed Control Systems

Assignment 2 - Lectures 3 and 4

Instructions:

- The assignments are individual. You may consult and discuss with your colleagues, but you need to provide independent answers.
- You may use Matlab/Python/Julia to solve the assignment. Many questions require you clearly to do so.
- Provide detailed answers, describing the steps you followed.
- Provide clear reports, typed, preferably using LaTex.
- Include with your report the **"in-depth" AI acknowledgement** from the ME Faculty. Not doing so will result in your report not being considered handed on time.
- Submit the reports digitally on Brightspace as indicated on the lectures.
- Include your code, for reproducibility of your results, in your Brightspace submission in a zip file, or through a link on your report. Make sure the report is in a separate PDF.
- The code will *only* be used to verify reproducibility in case of doubts. The grading will be performed based on the results described in the report.

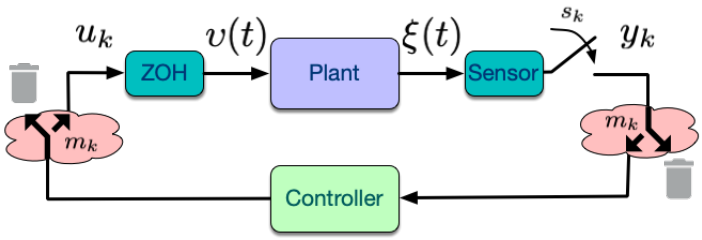

Figure 1: Schematic of a NCS including delays and packet-losses.

Consider a sample-data networked control system (NCS), as depicted in Figure 1. Assume that the plant dynamics are linear time-invariant given by:

$$\dot{\xi}(t) = A\xi(t) + Bv(t),$$

where the control signal v is a piece-wise constant signal resulting from the application of a controller in sampled-and-hold fashion, i.e. $v(t) = u_k,\ t \in [s_k,\ s_{k+1})$.

The system matrices are given by:

$$A = \begin{bmatrix} 0 & 0.5+c \\ 0.5+|a-b| & 1 \end{bmatrix},\ B = \begin{bmatrix} 1 \\ 0 \end{bmatrix},$$

where a is given by the first digit, b by the third digit, and c by the last digit of your student ID number. **This is the same plant from Assignment 1.**

We revisit the problem in Questions 3 of Assignment 1. We consider, as in Assignment 1, that the system is is subject to time varying delays taking three possible values $\tau_0 = 0$ (no delay), $\tau_1 = 0.5h$ and $\tau_2 = 1.5h$. Consider the **same controller K and implementation as in Assignment 1**.

First we assume a model for the delays dictating that τ_1 appears *a maximum of two consecutive times*, and τ_0 and τ_2 can only happen *isolated*. *E.g.* $\tau_1\tau_1\tau_2\tau_1\tau_1\tau_0\tau_2\tau_0$... is a valid sequence, while $\tau_1\tau_1\tau_2\tau_2\tau_0$... or $\tau_2\tau_0\tau_1\tau_1\tau_1$... are NOT valid sequences.

Question 1: (6p)

1. (2p) Create an ω-automaton that models this behavior.
2. (3p) Provide the LMIs needed to verify the stability of your closed-loop system.
3. (1p) Solve the LMIs to estimate the range of sampling intervals h for which the system is stable in this scenario. Can you relate to the results of Q3 in Assignment 1?

We consider now an alternative stochastic model for the delay patterns. Each delay τ_1 and τ_2 has a probability of repeating $p \leq 0.3$ and $q \leq 0.2$ of being followed by no delay (τ_0). After a no-delay τ_1 and τ_2 follow with equal probability.

Question 2: (7p)

1. (2p) Create a Markov Chain that models this behavior.
2. (3p) Choose a stochastic stability notion and provide LMIs to verify the stability of your closed-loop system.
3. (2p) Select 4 values of h that resulted in a stable system in Question 1. For which values of (p, q) is the system stable? Illustrate with figures and discuss the observed results.

Next, we assume that the system is affected by a **constant** small delay $\tau \in [0, h)$.

Question 3: (6p)

1. (3p) Employ the so-called Jordan form approach (as in Lecture 4) to construct a polytopic discrete-time model over-approximating the uncertain exact discrete-time closed-loop NCS dynamics.
2. (2p) Perform stability analyses, solving the relevant LMIs resulting from the previous question polytopic over-approximation. Employing this analysis, produce a plot illustrating combinations of (h, τ) as in Question 2 of Assignment 1.
3. (1p) Compare the resulting plot with the one from Assignment 1 Q2.2 and discuss any possible differences and the possible causes for such differences.

Finally we go back to the case with no delays to design an event-triggered controller.We consider once again your system being controlled by the first static controller you designed for your continuous-time system K.

Question 4: (6p)

1. (1p) Design a quadratic event-triggered condition

   $$\phi(\xi(t), \xi(s_k)) \leq \epsilon$$

   for the system to guarantee global practical stability of the closed-loop.
2. (2p) Select some value r and an appropriate ϵ such that $\exists T$, such that $\|\xi(T)\| < r$. Argue how you computed ϵ given r.
3. (2p) Simulate the resulting closed loop for various values of σ, the parameter controlling the convergence speed of the closed-loop, and initial conditions with identical norm. Compare the average amount of communications/sampling produced by each of the systems (different σ values) until entering the ball of radius r around the origin.
4. (1p) Comment on the results, if possible also comparing with the results from Assignment 1.

Hint: To simulate event-triggered control in continuous-time, you need to detect the triggering even while solving the ODE. The naïve way is to simulate at a very fine discrete-time rate, but this ends up being imprecise and computationally inefficient. ODE solvers in Python, Matlab, and Simulink, for example, have event-checking mechanisms that you can set up to, e.g., stop simulation at an state-dependent event. You can use this to stop simulation, update the control action, and restart the simulation in a loop. Here are some pointers for event detection:

- **Matlab**: any ODE solver, e.g., `ode45`, has an optional `options` input; look up `odeset` for implementation details (e.g., type `doc odeset` in the Matlab prompt and look for the"events" Section.)
- **Python**: similar to Matlab, `scipy` offers event-handling through an optional `events` argument. See the documentation for implementation details.
- **Simulink**: The Triggered subsystem is the recommended block to implement an event-based update of a signal.

If you are familiar with hybrid systems, particularly the jump-flow formalism, you may also consider the Matlab/Simulink toolbox HyEQ from USCB.